Climbing-Hold Detection with Superpixels

Code and workflow documentation

[常凱文 61575082H]

Computer Vision — Midterm, National Taiwan Normal University

November 2026

1 About this project

Given a picture of a climbing wall, the system finds the colored holds and outputs, for each hold, its *type*, its *position* (x, y, z) in meters and its *orientation* (roll, pitch, yaw). It uses the three OpenCV superpixel algorithms SLIC, LSC and SEEDS and compares them. There are two kinds of input; each has its own code folder, data and tuned values (the *baseline*).

	Synthetic walls	**Real walls**
Names	`synthetic_wall1` …`synthetic_wall4`	`real_wall1`, `real_wall2`
Scale and origin	red square (1 m, 0.1 m on narrow walls)	20 cm grid of bolt holes
Ground truth	`scene.json` from the simulator	hand labels (`labels.json`, `real_wall2`)
Type / depth z	hold color / stereo pair	nearest labelled color / none ($z = 0$)
Baseline files	`baseline/synthetic_wall_*.json`	`baseline/real_wall_*.json`

How to read this document. Commands to type in a terminal are in shaded boxes; file names, folder names and code are in `teal monospace`. All commands are run from the project root.

Two ways to run every script. (1) From a terminal, with options (`-h` lists them). (2) With the editor's Run button: every script that takes parameters has a block `Run-button settings` (at the top of a wall's `run.py`, at the bottom of the others) with the same options as plain variables, for example `MARKER = 0.1`. The button uses these values; an option typed in a terminal overrides them.

2 Quick start

Requirements: Python 3 with the *contrib* build of OpenCV (the superpixels are in `cv2.ximgproc`, which plain `opencv-python` lacks); `xelatex` only to build this document.

```
pip install opencv-contrib-python numpy matplotlib
```

2.1 Add a new synthetic wall

There are two ways to use a new wall. **Path A** only *tests* the existing baselines on it. **Path B** lets the wall take part in choosing the baselines. Do Path A first: once the wall is part of the choice, its score is no longer a fair test.

Step 0: add the files. Create `dataset/synthetic_wall4/` with:

File	Required	Notes
`wall.png` or `image.png`	yes	main camera view; the *whole* red square must be visible
`scene.json`	yes	ground truth; also gives the marker size (`scene.wall.originMarker.width`)
`stereo.png`	no	left eye | right eye side by side; without it $z = 0$
`camera.json`	no	stereo camera height and distance, read from the simulator page: `{"stereo": {"height_m": 0.7, "distance_m": 1.0}}`

Path A: held-out test ("how good are the frozen baselines on a wall they never saw?")

```
python3 src/pipelines/synthetic_wall/new_wall.py synthetic_wall4   # 1. create the wall's run script
python3 outputs/synthetic_wall4/scripts/run.py                     # 2. run SLIC, LSC and SEEDS on it
```

Step 1 checks the dataset folder and writes `outputs/synthetic_wall4/scripts/run.py`, a small wrapper you do not edit. Step 2 reads the baselines from `baseline/synthetic_wall_algorithm.json` *without changing*

them and writes `outputs/synthetic_wall4/<algo>/`. Open `detections.png` there (green = true hold found, red = false positive). The printed output has three parts: the new wall alone; a table of *every* wall (old walls are read from their saved `score.json`; read the `synthetic_wall4` rows, not the `all` rows); and the refreshed sweep summary, where only the new wall is computed (about 1 s) and the baseline is untouched. Options (terminal) or variables at the top of `run.py` (Run button): `--algo` / `ALGO = "slic"`, `--thresh`, `--min-sat` (override the baseline for this run), `--stereo-height`, `--stereo-distance` (no `camera.json`), `--marker` / `MARKER = 0.1` (only if `scene.json` lacks the marker size), `--no-pooled`, `--force`.

Path B: let the wall join the choice. This overwrites the baseline, so every number in this document can change.

```
python3 src/pipelines/synthetic_wall/run_all.py                    # sweep, pick, run, compare
python3 src/pipelines/synthetic_wall/run_all.py --keep-baseline    # same, but keep the baseline file
```

It does five steps: (1) **sweep**: only the new wall is computed, the others are cached, all are joined in `outputs/synthetic_wall_sweep/sweep_all.csv`; (2) **best**: the best setting per algorithm is chosen again (Section 5) and `synthetic_wall_algorithm.json` is overwritten; (3) **run** every wall with every algorithm; (4) **compare**: print the table; (5) **yaw**: redraw `synthetic_wall_yaw_examples.png`. With `--keep-baseline` step 2 is skipped; the summary then says "the sweep now prefers …" if a better setting exists. Afterwards rebuild the figures and this document (the numbers in the text are typed by hand):

```
python3 src/pipelines/make_report_figures.py
cd report && latexmk -xelatex main.tex
```

2.2 Add a real wall photo

```
python3 src/pipelines/real_wall/calibrate_grid.py real_wall3     # photo in dataset/real_wall3/image.png
python3 src/pipelines/real_wall/run_wall.py real_wall3           # all algorithms (or add: lsc)
```

Check `outputs/real_wall3/calibration.png`: the magenta grid must lie on the bolt holes and the median residual should be about 1 px. The detector then runs with the frozen real baseline. Only `real_wall2` has labels, so the score of a new photo means something only if it shows the same wall (labels are carried over by whole grid cells); otherwise judge `detections.png` by eye. To tune on another photo, label it in `labels.json` (`{"holds": [{"id": "hold_0", "pixel": [190, 120], "color": "green"}, ...]}`), set `labels_wall` and `test_wall` in `real_wall_parameters.json` and run the real sweep with `--save-best`.

2.3 Other tasks

```
python3 src/pipelines/run_all.py                              # everything, synthetic then real (about 1 minute)
python3 src/pipelines/synthetic_wall/sweep.py --save-best     # re-tune the synthetic baseline only
python3 src/pipelines/real_wall/sweep.py --save-best          # re-tune the real baseline only
python3 src/pipelines/synthetic_wall/sweep.py --replot        # redraw the plots from the saved csv
```

Fixed values (window, size limits, scale, stereo, yaw, grid) are edited by hand in `baseline/*_parameters.json`; the tuned values (size, T, S) are never edited by hand, only re-tuned. The sweep grid is `SIZES`, `THRESH`, `SATS` at the top of each `sweep.py`. A new superpixel algorithm is a new case in `segment()` and a name in `ALGOS` (`src/shared/superpixels.py`); every loop goes over `ALGOS`, so it gets its own baseline and folders.

2.4 Build this document (note for later)

The document is compiled with **XeLaTeX**, because the author line has Chinese characters: XeLaTeX can use any installed font through `fontspec`, here *Noto Serif CJK TC* (`\newfontfamily\cjkfont` in the preamble). This avoids the `ctex` package, which needs the large `texlive-lang-chinese`; plain `pdflatex` cannot typeset the Chinese characters. On Ubuntu/Debian:

```
sudo apt install texlive-xetex texlive-latex-recommended latexmk   # LaTeX compiler and the packages used here
sudo apt install fonts-noto-cjk                                    # Chinese/Japanese/Korean fonts
fc-list | grep "Noto Serif CJK TC"                                 # check that the font is found
```

Then compile; `latexmk` reruns XeLaTeX until the references are right, `-c` removes the helper files:

```
cd report
latexmk -xelatex main.tex                                  # writes main.pdf
latexmk -xelatex -interaction=nonstopmode main.tex         # same, never stops to ask on an error
latexmk -c                                                 # clean up .aux .log .xdv ...
```

If it stops with `File `xxx.sty' not found`, a LaTeX package is missing: type `X` and Enter to quit, find the Debian package with `apt-file search xxx.sty` and install it. Errors are explained in `report/main.log`. On Windows or macOS, install TeX Live or MiKTeX (both include XeLaTeX and `latexmk`) and any Noto CJK font.

3 Project layout

```
baseline/   synthetic_wall_parameters.json  real_wall_parameters.json    # fixed values, by hand
            synthetic_wall_algorithm.json   real_wall_algorithm.json    # tuned values, by the sweep
dataset/    synthetic_wall<N>/  real_wall<N>/                           # input only, never written
outputs/    <wall>/<algo>/  output.json score.json detections.png run.log
            <wall>/sweep.csv  <wall>/scripts/run.py
            synthetic_wall_sweep/  real_wall_sweep/                     # pooled sweep, ROC plots
src/shared/           paths superpixels candidates matching             # used by both kinds of wall
src/synthetic_wall/   baseline calibration evaluate orientation stereo
src/real_wall/        baseline grid_calibration detection
src/pipelines/        run_all make_report_figures                       # the runnable scripts
  synthetic_wall/     run_all sweep run_wall detect stereo_depth final_output new_wall
  real_wall/          run_all calibrate_grid sweep run_wall
```

Two rules: the two kinds of wall share *code* (`src/shared/`) but no tuned value; `dataset/` is only read, `outputs/` only written, and `baseline/` changes only when a sweep is run with `--save-best`.

4 How the pipeline works

Fig. 1 shows the whole pipeline; blue boxes are synthetic only, orange real only, green shared (`src/shared/`).

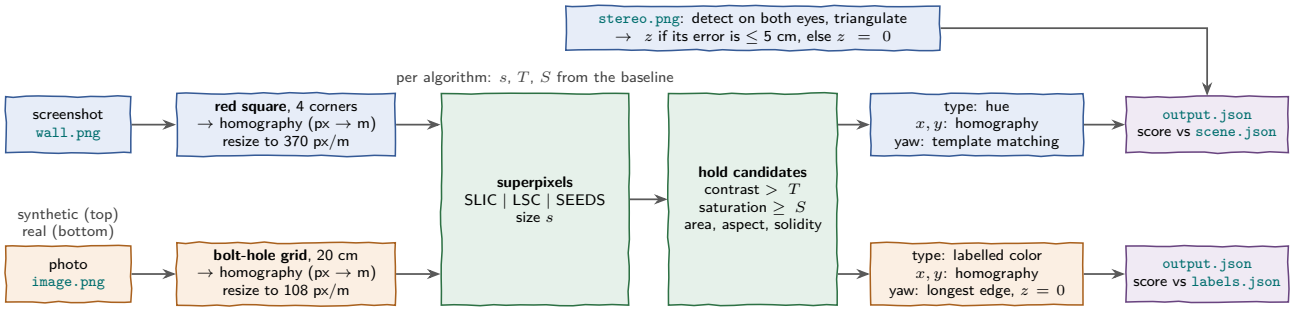
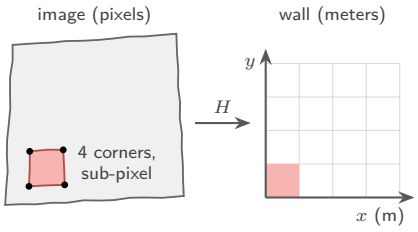
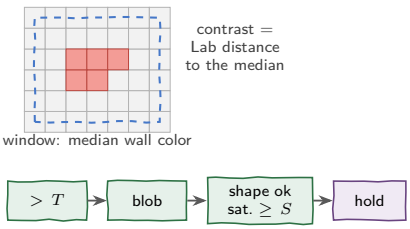


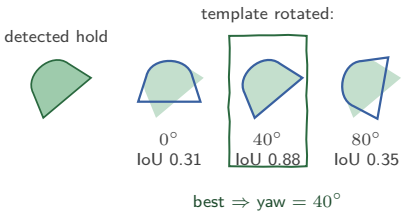
Figure 1: The pipeline. Each wall is first calibrated to meters and resized to a fixed resolution; the superpixels and the hold candidates are the same code for both kinds of wall; the description of a hold differs.



(a) **Calibration.** The square's corners give a homography; real walls use the 20 cm bolt-hole grid.



(b) **Hold candidates.** Superpixels that differ from the wall around them form blobs; shape and saturation remove shadows and volume faces.



(c) **Yaw.** The hold's silhouette is compared with one template hold rotated in 3° steps; the best overlap gives the yaw.

Figure 2: Three key steps of the detector.

Calibration. Synthetic: the red square's four corners give a homography pixel → meters (`synthetic_wall/calibration.py`); its size comes from `scene.json`. The contour corners lie on the centres of the edge pixels, which makes the square 1–2 px too small; `subpixel_quad()` enlarges them to the square's soft (anti-aliased) area. This matters for a small marker: on wall 4 (0.1 m marker, 65 px) it was a 2.7% scale error, up to 4 cm at the top of the wall. Real: small dark dots on plain board are found, given integer grid cells and grown into a lattice with a homography fit, 20 cm per cell (`real_wall/grid_calibration.py`). Each image is then resized to a fixed px/m, so a superpixel size in pixels means the same on every wall. The result is saved as `outputs/<wall>/calibration.npz` (the homography `H_pixel_to_m`, `px_per_m`, the origin and corners; real walls also the grid spacing, angle, wall extent and residuals). It is the first step of every run: `calibrate_grid.py` for real walls, `calibration.calibrate_wall()` for synthetic ones (called by `run_wall.run` and the sweep, once per wall before the parallel runs). Detection and the sweep only read it, with plain `np.load`.

Hold candidates (`hold_candidates()` in `shared/candidates.py`), the core of the detector. For each superpixel the code takes its mean Lab color and its Lab distance (the *contrast*) to the *median* color of the superpixels in a 36 px window, so the holds do not shift the wall color. Superpixels with contrast $> T$ are kept, cleaned by

opening/closing and grouped into blobs. A blob is a hold if its area, aspect ratio and solidity fit a hold and its saturation is at least S; the floor S removes shadows and gray volume faces.

Type, position, yaw, depth. Type is the nearest simulator hue (synthetic) or nearest labelled color in Lab (real). Position is the blob centroid through the homography. Synthetic yaw: the hold's silhouette is compared with a template hold rotated in 3° steps (intersection over union, `orientation.py`); real yaw: the direction of the longest straight edge. Roll and pitch are written as 0. Depth: the detector runs on both stereo eyes, holds are matched by color along the epipolar line and triangulated; since the error grows with the squared distance, z is used only if its expected error is ≤ 5 cm, otherwise $z = 0$ (`final_output.py`).

The three algorithms get a 3×3 blur and 10 iterations, single-threaded (multithreaded LSC is not deterministic). SLIC (SLICO) is k-means on (Lab, position) in a local window [1]; LSC is weighted k-means that approximates normalized cuts [2]; SEEDS moves block borders of a coarse grid to improve a color-histogram energy [3]. The size s is the region size for SLIC/LSC and gives area/s^2 superpixels for SEEDS. Fig. 3 shows the three on the same patch.

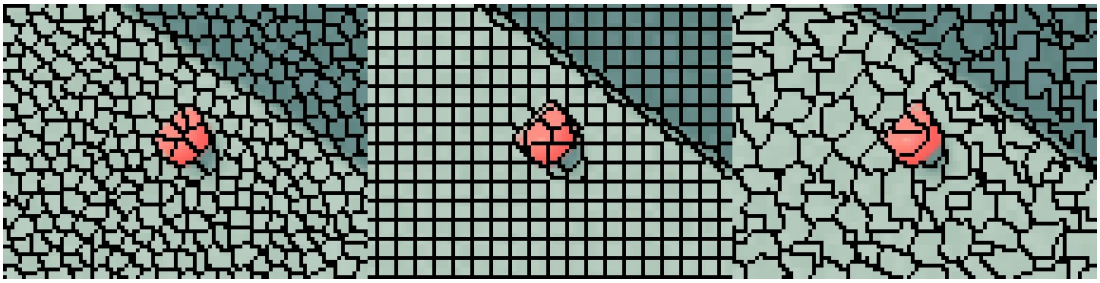


Figure 3: The same patch (a hold next to a pyramid) under SLIC, LSC and SEEDS, size 5. SLIC cells are compact and follow the hold; LSC is almost a regular grid; SEEDS gives irregular blocks.

Output (`outputs/<wall>/<algo>/output.json`), one entry per hold, meters and radians; origin at the bottom-left of the red square (synthetic) or the bottom-left bolt hole (real), x right, y up, z out of the wall:

```
{"type": "#F2554E", "position": {"x": -0.885, "y": 0.813, "z": 0.0},
 "orientation": {"roll": 0.0, "pitch": 0.0, "yaw": 0.681}}
```

Next to it: `score.json` (comparison with the ground truth), `detections.png`, `superpixels.jpg`, `mask.png` and `run.log`.

5 Choosing the parameters: the sweep

Three values are tuned per algorithm: the superpixel size s, the contrast threshold T and the saturation floor S. The sweep (`pipelines/*/sweep.py`) tries every combination and counts the result against the ground truth:

	Synthetic	Real
grid	3 algorithms × sizes 3–14 (7) × $T = 10 \dots 100$ (19) × $S \in \{0, 118\}$	the same, with $S \in \{0, 60, 118\}$
settings	798, each on 4 walls (walls 1–3 chose the current baseline)	1197, tuned on `real_wall2`, tested on `real_wall1`

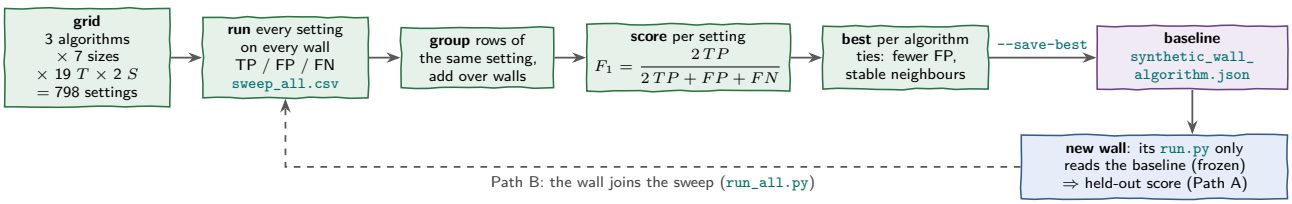


Figure 4: From the sweep to a baseline. A setting is one combination of algorithm, size s, threshold T and saturation floor S; it is scored on all walls together.

From sweep rows to a baseline (Fig. 4) (synthetic; the real sweep is the same on the labelled photo only):

1. **Count.** A detection within 10 cm of an unmatched true hold is a true positive (TP), any other is a false positive (FP), an unmatched true hold is a miss (FN) (`match()` in `shared/matching.py`).
2. **Group and add.** Rows with the same (algorithm, s, T, S) differ only in the wall; their TP, FP, FN are added (`pooled()`). Example: SLIC, 8, 40, 118 gives 9/0/1, 5/1/2, 6/0/4 on walls 1–3, total 20/1/7.

3. **Score.** Each group gets $F_1 = 2\,\mathrm{TP}/(2\,\mathrm{TP} + \mathrm{FP} + \mathrm{FN})$; above $40/48 = 0.83$.
4. **Pick.** Per algorithm the group with the best F_1 wins; ties go to fewer false positives, then to the higher F_1 of the neighbouring thresholds (`rank()`, `choose()`). The best of the three is `best_algorithm`.

ROC curve. The crucial parameter is T: each threshold gives one point (false positives, recall), and the line is the best recall reachable with at most k false positives (Fig. 5). A false-positive *rate* is not defined here, since there is no fixed set of non-holds.

Frozen baselines. The baseline files change only with `--save-best`. A wall's own `run.py` only reads them, so a wall added later is scored with values that never saw it (a held-out score).

Synthetic baseline (chosen on walls 1–3, 27 holds)						**Real baseline** (chosen on `real_wall2`)				
	s	T	S	found / FP	F_1		s	T	S	F_1
SLIC (best)	8	40	118	20 / 1	0.83	SLIC	4	60	60	0.74
LSC	3	10	0	19 / 1	0.81	LSC (best)	5	65	60	0.77
SEEDS	3	30	0	21 / 6	0.78	SEEDS	3	70	60	0.73

SLIC leads LSC by 0.02, but on 27 holds one hold is about 0.03 of F_1, so the ranking is not significant. The optimum is also unstable: the calibration fix changed the scale by less than 1% and moved SLIC's best size from 5 to 8 (before: SLIC 5/40/118, LSC 3/15/0, SEEDS 3/30/118, F_1 0.80/0.80/0.77). Superpixel time at size 8 on the four walls: SEEDS 2.5 s, SLIC 5.1 s, LSC 16.3 s.

6 Results

6.1 Synthetic walls

Walls 1–3 chose the baselines; `synthetic_wall4` was added afterwards with Path A, so it is a held-out test.

SLIC, per wall					**Every algorithm**		
Wall	found, FP	position	type	yaw (mean / med.)		4 walls: found, FP	wall 4 F_1
1	9 / 10, 0	1.0 cm	9 / 9	13° / 3°	SLIC	27 / 37, 1	0.82
2	5 / 7, 1	1.6 cm	5 / 5	60° / 72°	LSC	24 / 37, 1	0.67
3	6 / 10, 0	0.7 cm	6 / 6	5° / 3°	SEEDS	26 / 37, 6	0.67
4 (held out)	7 / 10, 0	1.1 cm	7 / 7	28° / 3°			
all four	27 / 37, 1	1.0 cm	27 / 27	24° / 5°			

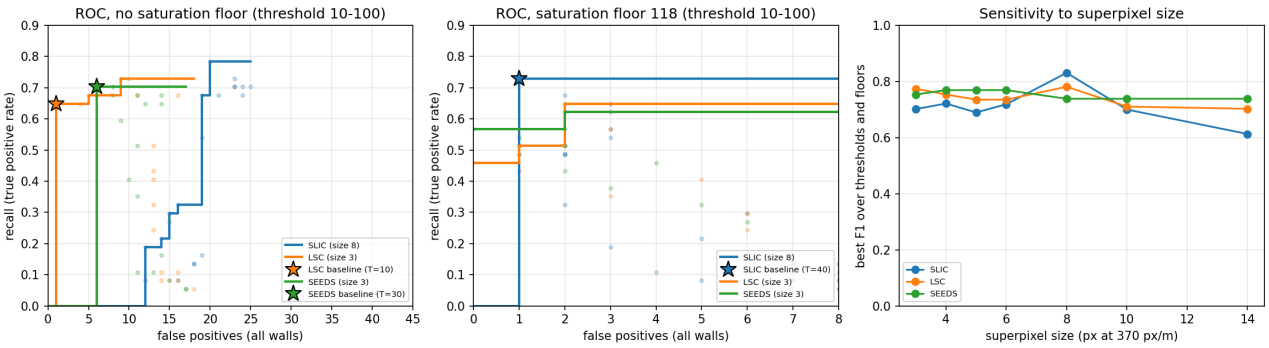


Figure 5: Left, middle: ROC curves from sweeping $T = 10 \ldots 100$, pooled over the four walls, each algorithm at its baseline size (line: best recall with at most k false positives; dots: single thresholds; stars: baselines). Right: best F_1 against the superpixel size.

- **Held-out wall.** With the frozen baselines SLIC finds 7 of 10 holds on wall 4 with no false positive (F_1 0.82, the same as on the tuning walls); LSC and SEEDS find 5 (F_1 0.67).
- **The saturation floor matters most.** Without it, reaching 20 of 37 holds costs SLIC 19 false positives and SEEDS 6; with it SLIC finds 27 with one (Fig. 5). LSC is the exception: it reaches 20 holds with one false positive without the floor, so its baseline does not use it.
- T **is fairly robust:** SLIC keeps 23–27 holds with 1–3 false positives for $T = 35 \ldots 50$ and drops to 12 holds at $T = 60$ (Fig. 6). **The size matters little** between 3 and 10 px (best F_1 0.69–0.78 for all three), except a peak of SLIC at 8 px (0.83); at 14 px SLIC falls to 0.61, where a superpixel is as large as a hold.

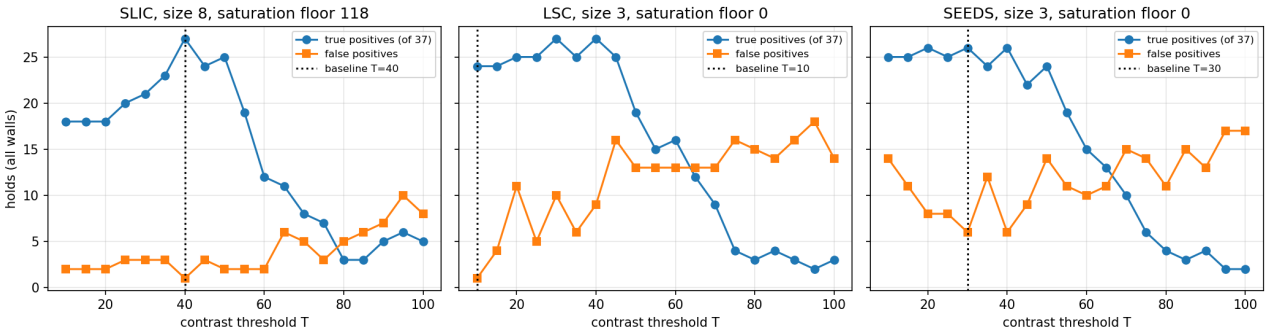


Figure 6: Sensitivity to the contrast threshold T, each algorithm at its baseline size and saturation floor (four walls).

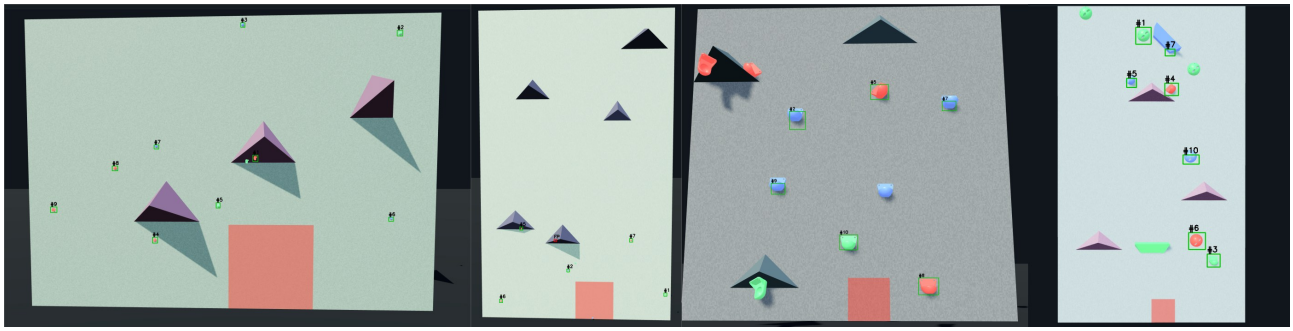


Figure 7: SLIC detections on the four synthetic walls, wall 4 (held out) on the right. Green: matched to a true hold; red: false positive; uncircled holds are misses.

- **Type is always right** (color) for the found holds; the position error is 0.7–1.6 cm on walls 1–3 and 1.1 cm on wall 4. **Yaw** is good on walls 1 and 3 (median 3°) and fails on wall 2 (holds only 6–7 px wide). **Depth** is about 1–2 cm with the camera at 0.75 m (wall 1); at 1–2 m the expected error exceeds 5 cm and $z = 0$.
- **Wall 4 brings new hold shapes.** Walls 1–3 contain only `microban_hold`; wall 4 adds `fingerpocket_hold` (a ball with holes) and `sidepull_hold` (a long bar). The fixed shape filters (aspect ≤ 2.5, solidity ≥ 0.85) were set for microban holds, so the long sidepulls are split or rejected and fingerpockets with dark holes can fail the solidity test: SLIC's misses are one sidepull and two fingerpockets, and its only large position error (4.5 cm) is the other sidepull, of which only one end was found. The yaw template is a microban hold: the microbans get 0–3° (one 45°), the other shapes are not meaningful (63–87° on two of four).

6.2 Real walls

`real_wall2` (left third of the wall, 100 holds labelled by hand) is the tuning photo; `real_wall1` (whole wall, other camera position) is the test photo, scored inside the region `real_wall2` shows. The grid calibration fits 420 and 238 holes with a median residual of 1 px.

Algorithm	`real_wall2` (tuning) found / FP	F_1	`real_wall1` (test) found / FP	F_1	position
SLIC	66 / 13	0.74	57 / 15	0.66	3.4–3.9 cm
LSC	**70 / 13**	**0.77**	**60 / 14**	**0.69**	3.8–3.9 cm
SEEDS	65 / 14	0.73	53 / 13	0.64	3.7–4.3 cm

LSC is best on both photos, SLIC is close, SEEDS last (Fig. 8); all three choose $S = 60$. The type agrees with the labels for 76–77% of the holds LSC finds, mostly mixing brown, tan and yellow. The optimum is flat: a 0.1% change of the working scale moved the best thresholds by 5–20 and F_1 by up to 0.06, so differences of a few hundredths are not significant. Yaw has no ground truth on the real walls.

7 Limits and troubleshooting

- **Little data and optimistic tuning scores.** 27 synthetic holds chose the baseline and are also scored; an earlier leave-one-wall-out check (before the calibration fix) lowered SLIC's F_1 from 0.80 to about 0.70. The fair number is wall 4 (F_1 0.82 for SLIC, but from only 10 holds). The real labels come from one photo, and the test photo shows the same holds.

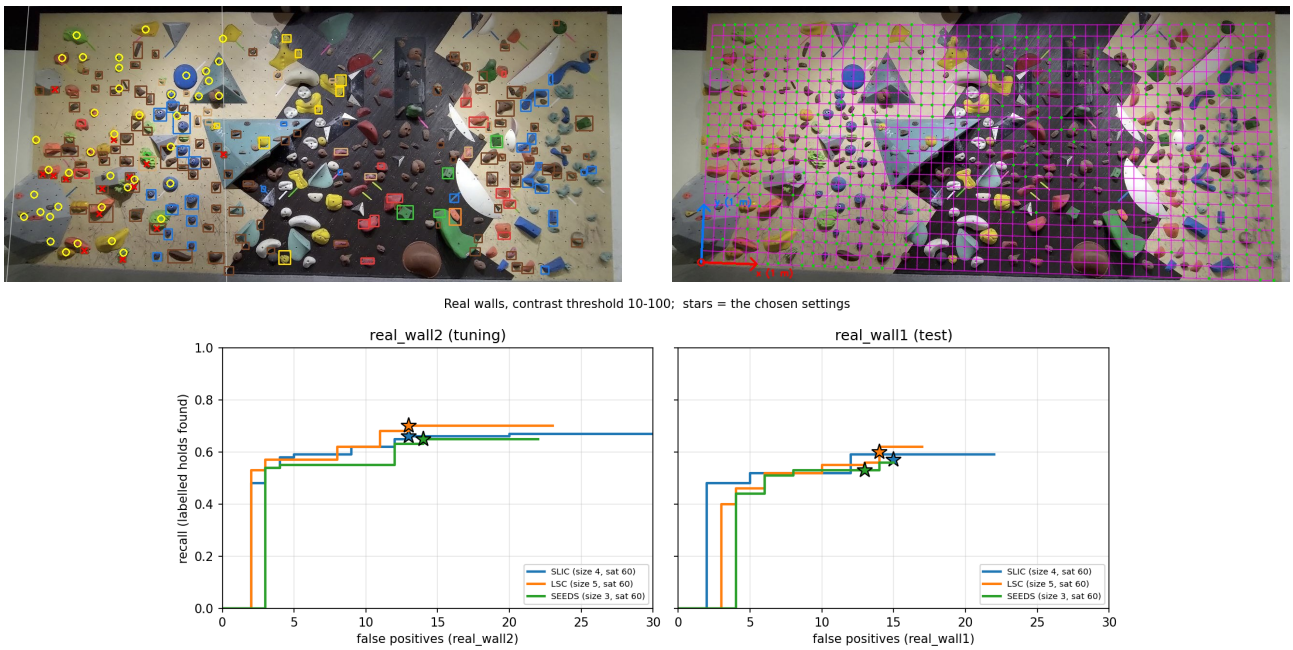


Figure 8: Top left: `real_wall1` with LSC (box color = type, red cross = false positive, yellow circle = missed labelled hold). Top right: its grid calibration (magenta 20 cm grid, green bolt holes used). Bottom: ROC on the real photos ($T = 10 \dots 100$); stars: the chosen settings.

- **Holds on volumes** are mostly missed, and a found one gets $z = 0$ and roll $= 0$; roll and pitch are never estimated. **Small holds** (5–6 px before resizing) are the weakest case, also for yaw. The yaw template comes from one hold. Real walls are treated as one plane with $z = 0$.
- **Hold type means color here.** In `scene.json` the `type` field is the hold *shape* (`microban_hold`, `fingerpocket_hold`, `sidepull_hold`); the output gives the color, which was the only difference between holds on walls 1–3. Classifying shapes, a template per shape and shape-aware filters are the next step; a wall used to develop them would no longer be a held-out test.

Symptom	Fix
`red square not found`	export the screenshot again with the whole square visible
0 holds on a narrow synthetic wall	check `originMarker.width` in `scene.json` (0.1 m) or pass `--marker`
`camera.json` not found	add it, or pass `--stereo-height` and `--stereo-distance`
`only N grid holes found`	use a larger photo or lower `dot_contrast` in `real_wall_parameters.json`
a sweep does not rerun after an edit	add `--force`
no module `cv2.ximgproc`	install `opencv-contrib-python` instead of `opencv-python`

References

[1] R. Achanta et al., "SLIC superpixels compared to state-of-the-art superpixel methods," IEEE TPAMI, 2012.

[2] Z. Li and J. Chen, "Superpixel segmentation using linear spectral clustering," CVPR, 2015.

[3] M. Van den Bergh et al., "SEEDS: superpixels extracted via energy-driven sampling," ECCV, 2012.